JavaScript

A Complete Guide from Zero to Hero

Comprehensive Learning Resource

Topics Covered: Fundamentals to Advanced Concepts

Table of Contents

01 Introduction to JavaScript

02 Setting Up the Environment

03 Variables & Data Types

04 Operators

05 Control Flow

06 Loops

07 Functions

08 Arrays

09 Objects

10 Strings & Template Literals

11 DOM Manipulation

12 Events

13 ES6+ Modern Features

14 Asynchronous JavaScript

15 Error Handling

16 Modules

17 Best Practices

1. Introduction to JavaScript

JavaScript is a versatile, high-level programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS. It enables interactive web pages and is an essential part of web applications.

What is JavaScript?

JavaScript is a **lightweight, interpreted** programming language with first-class functions. While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js.

> **KEY FACT**
> JavaScript was created in just **10 days** by Brendan Eich in 1995 while he was working at Netscape.

What Can JavaScript Do?

- Modify HTML content and attributes
- Change CSS styles dynamically
- Validate form data before submission
- Create cookies and store data locally
- Build full-stack applications with Node.js
- Create mobile apps with React Native
- Develop desktop apps with Electron

First JavaScript Program

```
// This is your first JavaScript program
console.log("Hello, World!");
```

Where to Insert JavaScript

JavaScript can be placed in the HTML document in three ways:

1. Internal JavaScript (in HTML)

```
<script>
console.log("Hello from internal JS");
</script>
```

2. External JavaScript File

```
<script src="script.js"></script>
```

3. Inline (not recommended)

```
<button onclick="alert('Clicked!')">Click Me</button>
```

2. Setting Up the Environment

Tools You Need

- **Browser:** Chrome, Firefox, Edge, or Safari (Chrome recommended)
- **Code Editor:** VS Code, Sublime Text, or WebStorm
- **Browser DevTools:** Press F12 to open console

Using Browser Console

The easiest way to start learning JavaScript is through the browser console:

- Open any web page
- Press **F12** or right-click -> Inspect
- Click the **Console** tab
- Type JavaScript commands and press Enter

Node.js Setup

For running JavaScript outside the browser:

```
// Install Node.js from nodejs.org
// Then run files using terminal:
node myfile.js
```

NOTE
When using the browser console, you have access to the **document** and **window** objects. In Node.js, these are not available.

3. Variables & Data Types

Declaring Variables

JavaScript has three ways to declare variables:

Keyword	Scope	Can Reassign?	Can Redeclare?
var	Function / Global	Yes	Yes

Keyword	Scope	Can Reassign?	Can Redeclare?
let	Block	Yes	No
const	Block	No	No

```
let name = "John"; // Can be reassigned
const PI = 3.14159; // Cannot be reassigned
var oldWay = "avoid"; // Old way, function scoped
```

BEST PRACTICE
Always use **const** by default. Use **let** only when you need to reassign. Avoid **var** entirely.

Data Types

JavaScript has 8 basic data types:

Primitive Types (7)

```
let str = "Hello"; // String
let num = 42; // Number
let big = 9007199254740991n; // BigInt
let bool = true; // Boolean
let nothing = null; // Null
let notDefined; // Undefined
let sym = Symbol("id"); // Symbol
```

Object Type (1)

```
let person = {
name: "Alice",
age: 25
};
```

Type Checking

```
typeof "hello"; // "string"
typeof 42; // "number"
typeof true; // "boolean"
typeof {}; // "object"
typeof null; // "object" (historical bug!)
typeof function(){}; // "function"
```

Type Conversion

```
// String to Number
Number("42"); // 42
parseInt("42px"); // 42
parseFloat("3.14"); // 3.14
```

```
// Number to String
String(42); // "42"
(42).toString(); // "42"

// Implicit conversion
"5" + 3; // "53" (string concatenation)
"5" - 3; // 2 (numeric subtraction)
```

4. Operators

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus	5 % 3	2
**	Exponentiation	2 ** 3	8

Assignment Operators

```
let x = 10;
x += 5; // x = x + 5 -> 15
x -= 3; // x = x - 3 -> 12
x *= 2; // x = x * 2 -> 24
x /= 4; // x = x / 4 -> 6
x %= 4; // x = x % 4 -> 2
```

Comparison Operators

```
5 == "5"; // true (loose equality)
5 === "5"; // false (strict equality)
5 != "5"; // false
5 !== "5"; // true
10 > 5; // true
10 <= 5; // false
```

IMPORTANT
Always use **===** and **!==** instead of **==** and **!=** to avoid unexpected type coercion.

Logical Operators

```
let a = true, b = false;

a && b; // false (AND)
a || b; // true (OR)
!a; // false (NOT)

// Short-circuit evaluation
let name = user || "Guest"; // Default value pattern
```

Ternary Operator

```
let age = 18;
let status = age >= 18 ? "Adult" : "Minor";
```

5. Control Flow

if / else if / else

```
let score = 85;

if (score >= 90) {
console.log("Grade: A");
} else if (score >= 80) {
console.log("Grade: B");
} else if (score >= 70) {
console.log("Grade: C");
} else {
console.log("Grade: F");
}
```

switch Statement

```
let day = 3;
let dayName;

switch (day) {
case 1:
dayName = "Monday";
break;
case 2:
dayName = "Tuesday";
break;
```

```
case 3:
dayName = "Wednesday";
break;
default:
dayName = "Unknown";
}
```

Truthy and Falsy Values

Falsy: false, 0, "" (empty string), null, undefined, NaN

Truthy: true, any non-zero number, any non-empty string, objects and arrays, "0" and "false"

6. Loops

for Loop

```
for (let i = 0; i < 5; i++) {
console.log("Iteration: " + i);
}
// Output: 0, 1, 2, 3, 4
```

while Loop

```
let count = 0;
while (count < 5) {
console.log(count);
count++;
}
```

do...while Loop

```
let num = 0;
do {
console.log(num);
num++;
} while (num < 5);
```

for...of (Iterables)

```
const fruits = ["Apple", "Banana", "Cherry"];

for (const fruit of fruits) {
console.log(fruit);
}
```

for...in (Objects)

```
const person = { name: "John", age: 30 };

for (const key in person) {
console.log(`${key}: ${person[key]}`);
}
```

Loop Control

```
for (let i = 0; i < 10; i++) {
if (i === 3) continue; // Skip iteration
if (i === 7) break; // Exit loop
console.log(i);
}
```

7. Functions

Function Declaration

```
function greet(name) {
return "Hello, " + name + "!";
}

console.log(greet("Alice")); // "Hello, Alice!"
```

Function Expression

```
const add = function(a, b) {
return a + b;
};

console.log(add(5, 3)); // 8
```

Arrow Functions (ES6)

```
// Single parameter - parentheses optional
const square = x => x * x;

// Multiple parameters
const multiply = (a, b) => a * b;

// Multiple statements
const greet = (name) => {
const message = "Hello, " + name;
return message;
};
```

Default Parameters

```
function greet(name = "Guest", greeting = "Hello") {
return `${greeting}, ${name}!`;
}

greet(); // "Hello, Guest!"
greet("Alice"); // "Hello, Alice!"
```

Rest Parameters

```
function sum(...numbers) {
return numbers.reduce((total, num) => total + num, 0);
}

sum(1, 2, 3, 4); // 10
```

Higher-Order Functions

```
function createMultiplier(factor) {
return function(number) {
return number * factor;
};
}

const double = createMultiplier(2);
console.log(double(5)); // 10
```

Scope & Closures

```
function counter() {
let count = 0;
return {
increment: () => ++count,
decrement: () => --count,
getCount: () => count
};
}

const myCounter = counter();
myCounter.increment(); // 1
myCounter.increment(); // 2
```

8. Arrays

Creating Arrays

```
const fruits = ["Apple", "Banana", "Cherry"];
const numbers = new Array(5); // [empty x5]
const mixed = [1, "two", true, null];
```

Accessing Elements

```
fruits[0]; // "Apple"
fruits.length; // 3
fruits[fruits.length - 1]; // "Cherry" (last element)
```

Common Array Methods

```
const arr = [1, 2, 3];

// Adding/Removing
arr.push(4); // [1,2,3,4] - add to end
arr.pop(); // [1,2,3] - remove from end
arr.unshift(0); // [0,1,2,3] - add to start
arr.shift(); // [1,2,3] - remove from start

// Searching
arr.indexOf(2); // 1
arr.includes(2); // true
arr.find(x => x > 1); // 2

// Transforming
arr.map(x => x * 2); // [2,4,6]
arr.filter(x => x > 1); // [2,3]
arr.reduce((a, b) => a + b, 0); // 6

// Other
arr.slice(1, 3); // [2,3] - extract portion
arr.splice(1, 1); // removes 1 element at index 1
arr.sort(); // sorts in place
arr.reverse(); // reverses in place
arr.join("-"); // "1-2-3"
```

Array Destructuring

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];
// first = 1, second = 2, rest = [3,4,5]
```

Spread Operator

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const combined = [...arr1, ...arr2]; // [1,2,3,4]

const copy = [...arr1]; // Shallow copy
```

9. Objects

Creating Objects

```
// Object literal
const person = {
firstName: "John",
lastName: "Doe",
age: 30,
isStudent: false,
fullName: function() {
return `${this.firstName} ${this.lastName}`;
}
};
```

Accessing Properties

```
// Dot notation
person.firstName; // "John"

// Bracket notation
person["lastName"]; // "Doe"

// Dynamic property access
const prop = "age";
person[prop]; // 30
```

Modifying Objects

```
// Add property
person.email = "john@example.com";

// Update property
person.age = 31;

// Delete property
delete person.isStudent;

// Check property existence
"email" in person; // true
person.hasOwnProperty("age"); // true
```

Object Destructuring

```
const { firstName, age, country = "USA" } = person;
// firstName = "John", age = 30, country = "USA"

// Nested destructuring
const user = { name: "Alice", address: { city: "NYC" } };
```

```
const { address: { city } } = user; // city = "NYC"
```

Object Methods

```
Object.keys(person); // ["firstName", "lastName", ...]
Object.values(person); // ["John", "Doe", 30, ...]
Object.entries(person); // [["firstName", "John"], ...]

// Merge objects
const merged = { ...obj1, ...obj2 };
const merged2 = Object.assign({}, obj1, obj2);
```

Classes (ES6)

```
class Animal {
constructor(name) {
this.name = name;
}

speak() {
console.log(`${this.name} makes a sound.`);
}
}

class Dog extends Animal {
constructor(name, breed) {
super(name);
this.breed = breed;
}

speak() {
console.log(`${this.name} barks!`);
}
}

const dog = new Dog("Buddy", "Golden Retriever");
dog.speak(); // "Buddy barks!"
```

10. Strings & Template Literals

String Creation

```
const single = 'Hello';
const double = "World";
const template = `Hello ${double}`; // Template literal
```

String Methods

```
let text = " Hello World ";

text.length; // 15
text.toUpperCase(); // " HELLO WORLD "
text.toLowerCase(); // " hello world "
text.trim(); // "Hello World"
text.trimStart(); // "Hello World "
text.trimEnd(); // " Hello World"

let str = "JavaScript";
str.charAt(0); // "J"
str.indexOf("Script"); // 4
str.includes("Script"); // true
str.startsWith("Java"); // true
str.endsWith("Script"); // true
str.slice(0, 4); // "Java"
str.substring(4, 10); // "Script"
str.replace("Java", "Type"); // "TypeScript"
str.split(""); // ["J","a","v","a",...]
```

Template Literals (Backticks)

```
const name = "Alice";
const age = 25;

// Variable interpolation
const greeting = `Hello, my name is ${name} and I am ${age} years old.`;

// Multi-line strings
const html = `
<div>
<h1>${name}</h1>
<p>Age: ${age}</p>
</div>
`;

// Expression evaluation
const result = `Sum: ${2 + 3}`; // "Sum: 5"
```

11. DOM Manipulation

What is the DOM?

The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content.

Selecting Elements

```
// By ID
const element = document.getElementById("myId");

// By class name
const elements = document.getElementsByClassName("myClass");

// By tag name
const divs = document.getElementsByTagName("div");

// CSS Selectors (modern)
const first = document.querySelector(".myClass");
const all = document.querySelectorAll(".myClass");
```

Modifying Content

```
const element = document.getElementById("demo");

// Text content
element.textContent = "New text";

// HTML content
element.innerHTML = "<strong>Bold text</strong>";

// Input value
const input = document.querySelector("input");
input.value = "New value";
```

Modifying Styles

```
const box = document.getElementById("box");

// Inline styles
box.style.backgroundColor = "red";
box.style.width = "200px";
box.style.display = "none";

// CSS classes
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("visible");
box.classList.contains("active"); // true/false
```

Creating & Removing Elements

```
// Create element
const newDiv = document.createElement("div");
newDiv.textContent = "I am new!";
newDiv.className = "new-element";
```

```
// Append to parent
const parent = document.getElementById("container");
parent.appendChild(newDiv);

// Insert before another element
parent.insertBefore(newDiv, parent.firstChild);

// Modern methods
parent.append(newDiv); // Append at end
parent.prepend(newDiv); // Append at start
parent.before(newDiv); // Insert before parent
parent.after(newDiv); // Insert after parent

// Remove element
newDiv.remove();
```

12. Events

Event Listeners

```
const button = document.getElementById("myBtn");

// Add event listener
button.addEventListener("click", function() {
console.log("Button clicked!");
});

// With arrow function
button.addEventListener("click", () => {
console.log("Clicked!");
});

// Remove event listener
function handleClick() {
console.log("Once");
}
button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick);
```

Common Events

Event	Description
click	Element is clicked
dblclick	Element is double-clicked

Event	Description
mouseover	Mouse enters element
mouseout	Mouse leaves element
keydown	Key is pressed down
keyup	Key is released
submit	Form is submitted
change	Input value changes
load	Page/resource finishes loading
resize	Window is resized
scroll	Page is scrolled
focus	Element receives focus
blur	Element loses focus

The Event Object

```
button.addEventListener("click", function(event) {
event.preventDefault(); // Prevent default action
event.stopPropagation(); // Stop event bubbling

console.log(event.target); // Element that triggered event
console.log(event.type); // Type of event
console.log(event.clientX); // Mouse X position
console.log(event.clientY); // Mouse Y position
});
```

Event Delegation

```
// Instead of adding listener to each item
const list = document.getElementById("myList");

list.addEventListener("click", function(e) {
if (e.target.tagName === "LI") {
console.log("List item clicked:", e.target.textContent);
}
});
```

13. ES6+ Modern Features

let and const

```
let count = 0; // Block-scoped, can reassign
const PI = 3.14; // Block-scoped, cannot reassign
const arr = []; // Can modify contents, not reference
```

Arrow Functions

```
// Traditional function
function add(a, b) {
return a + b;
}

// Arrow function
const add = (a, b) => a + b;

// With single parameter
const square = x => x * x;

// With block body
const greet = name => {
const msg = `Hello, ${name}`;
return msg;
};
```

Template Literals

```
const name = "World";
const message = `Hello, ${name}!`;

// Multi-line
const html = `
<div>
<h1>Title</h1>
<p>Content</p>
</div>
`;
```

Destructuring

```
// Array destructuring
const [a, b, c] = [1, 2, 3];

// Object destructuring
const { name, age } = { name: "John", age: 30 };

// With default values
```

```
const { x = 0, y = 0 } = { x: 10 };

// Nested destructuring
const { user: { email } } = { user: { email: "test@test.com" } };
```

Spread and Rest Operators

```
// Spread (expand)
const arr1 = [1, 2];
const arr2 = [3, 4];
const combined = [...arr1, ...arr2]; // [1,2,3,4]

const obj1 = { a: 1 };
const obj2 = { b: 2 };
const merged = { ...obj1, ...obj2 }; // {a:1, b:2}

// Rest (collect)
function sum(...numbers) {
return numbers.reduce((a, b) => a + b, 0);
}

const [first, ...rest] = [1, 2, 3, 4];
```

Default Parameters

```
function greet(name = "Guest", greeting = "Hello") {
return `${greeting}, ${name}!`;
}

greet(); // "Hello, Guest!"
greet("Alice"); // "Hello, Alice!"
greet("Bob", "Hi"); // "Hi, Bob!"
```

Classes

```
class Person {
constructor(name, age) {
this.name = name;
this.age = age;
}

greet() {
return `Hello, I am ${this.name}`;
}

static isAdult(age) {
return age >= 18;
}
}
```

```
class Student extends Person {
constructor(name, age, grade) {
super(name, age);
this.grade = grade;
}
}
```

Modules (import/export)

```
// math.js
export const PI = 3.14159;
export function add(a, b) {
return a + b;
}
export default class Calculator {}

// main.js
import Calculator, { PI, add } from "./math.js";
import * as math from "./math.js";
```

Promises

```
const promise = new Promise((resolve, reject) => {
if (success) {
resolve("Data loaded");
} else {
reject("Error occurred");
}
});

promise
.then(result => console.log(result))
.catch(error => console.error(error))
.finally(() => console.log("Done"));
```

Async/Await

```
async function fetchData() {
try {
const response = await fetch("https://api.example.com/data");
const data = await response.json();
return data;
} catch (error) {
console.error("Error:", error);
}
}
```

14. Asynchronous JavaScript

Callbacks

```
function fetchData(callback) {
setTimeout(() => {
callback("Data received");
}, 1000);
}

fetchData((data) => {
console.log(data);
});
```

Promises in Depth

```
const promise1 = Promise.resolve(42);
const promise2 = Promise.reject("Error");

// Promise.all - waits for all
Promise.all([promise1, promise2])
.then(results => console.log(results))
.catch(error => console.error(error));

// Promise.race - first to settle
Promise.race([promise1, promise2])
.then(result => console.log(result));

// Promise.allSettled - waits for all regardless of outcome
Promise.allSettled([promise1, promise2])
.then(results => console.log(results));
```

Async/Await Patterns

```
// Sequential execution
async function getUserData() {
const user = await fetchUser();
const posts = await fetchPosts(user.id);
return { user, posts };
}

// Parallel execution
async function getData() {
const [users, posts] = await Promise.all([
fetchUsers(),
fetchPosts()
]);
return { users, posts };
}
```

Fetch API

```
// GET request
fetch("https://api.example.com/users")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error("Error:", error));

// POST request
fetch("https://api.example.com/users", {
method: "POST",
headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({ name: "John", age: 30 })
});
```

15. Error Handling

try...catch...finally

```
try {
// Code that might throw an error
const result = riskyOperation();
console.log(result);
} catch (error) {
// Handle the error
console.error("An error occurred:", error.message);
} finally {
// Always executes
console.log("Cleanup complete");
}
```

Throwing Errors

```
function divide(a, b) {
if (b === 0) {
throw new Error("Cannot divide by zero");
}
return a / b;
}

try {
divide(10, 0);
} catch (error) {
console.error(error.message); // "Cannot divide by zero"
}
```

Custom Error Classes

```
class ValidationError extends Error {
constructor(message) {
super(message);
this.name = "ValidationError";
}
}

function validateAge(age) {
if (age < 0) {
throw new ValidationError("Age cannot be negative");
}
}
```

Error Handling in Async Code

```
// With async/await
async function fetchData() {
try {
const response = await fetch("/api/data");
if (!response.ok) {
throw new Error(`HTTP error! status: ${response.status}`);
}
const data = await response.json();
return data;
} catch (error) {
console.error("Fetch failed:", error);
return null;
}
}

// With promises
fetchData()
.then(data => console.log(data))
.catch(error => console.error(error));
```

16. Modules

Exporting

```
// Named exports
export const PI = 3.14159;
export function add(a, b) {
return a + b;
}
export class Calculator {}

// Default export (one per module)
export default function main() {
```

```
console.log("Main function");
}
```

Importing

```
// Import specific exports
import { PI, add } from "./math.js";

// Import with alias
import { add as sum } from "./math.js";

// Import default
import main from "./math.js";

// Import all as namespace
import * as math from "./math.js";
math.add(1, 2);

// Dynamic import
const module = await import("./math.js");
```

Module Types

Type	Extension	Usage
ES Modules	.js (with type="module")	Modern browsers, Node.js
CommonJS	.cjs	Node.js (older)
AMD	.js	RequireJS (legacy)
UMD	.js	Universal (legacy)

17. Best Practices

Code Style

- Use **const** by default, **let** when needed, never **var**
- Use strict equality **===** and **!==**
- Use meaningful variable names
- Keep functions small and focused
- Use camelCase for variables/functions, PascalCase for classes

- Use UPPER_SNAKE_CASE for constants

Performance Tips

```
// Cache DOM queries
const element = document.getElementById("myId");
// Use element multiple times instead of querying each time

// Use document fragments for multiple insertions
const fragment = document.createDocumentFragment();
for (let i = 0; i < 100; i++) {
const div = document.createElement("div");
fragment.appendChild(div);
}
document.body.appendChild(fragment);

// Debounce expensive operations
function debounce(func, wait) {
let timeout;
return function(...args) {
clearTimeout(timeout);
timeout = setTimeout(() => func.apply(this, args), wait);
};
}
```

Security Best Practices

- Never use **eval()** - it executes arbitrary code
- Sanitize user input before displaying
- Use **textContent** instead of **innerHTML** for user content
- Validate data on both client and server
- Use HTTPS for all network requests
- Keep dependencies updated

Modern JavaScript Checklist

```
// Use modern syntax
const user = { name, age }; // Shorthand properties
const { name } = user; // Destructuring
const arr = [...oldArr]; // Spread for copying

// Use optional chaining
const city = user?.address?.city;

// Use nullish coalescing
const value = input ?? "default";

// Use template literals
```

```
const msg = `Hello, ${name}`;

// Use async/await over callbacks
async function loadData() {
const data = await fetch("/api/data");
return data.json();
}
```

Debugging Tips

- Use **console.log()** for quick checks
- Use **console.table()** for arrays/objects
- Use **debugger;** statement for breakpoints
- Use browser DevTools for step-through debugging
- Use **console.time()** and **console.timeEnd()** for performance
- Use TypeScript for catching errors at compile time

KEEP LEARNING
JavaScript is constantly evolving. Stay updated with new features by following MDN Web Docs, JavaScript Weekly, and the TC39 proposals.